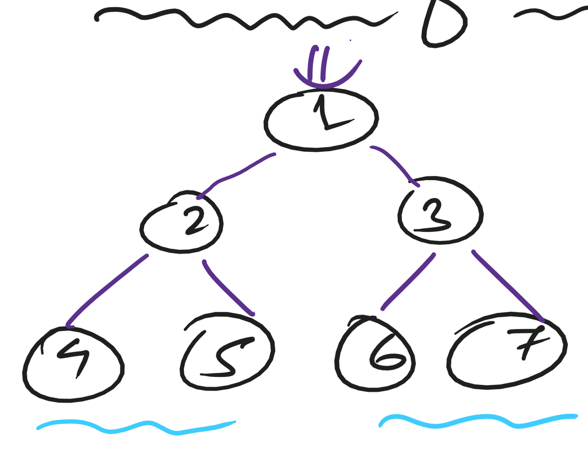
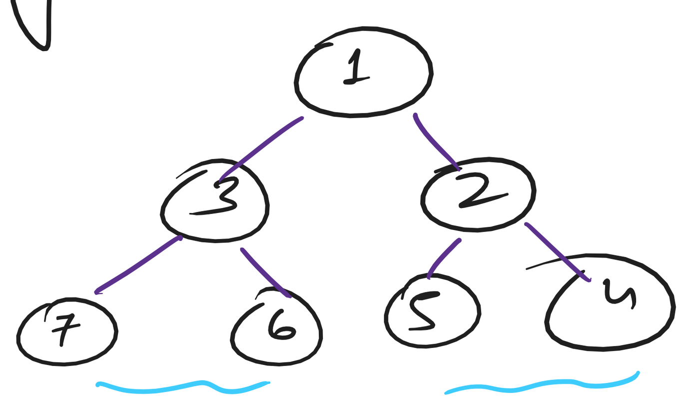
Mirror of a Binary Tree



4,2,5,1,6,3,7
In-order

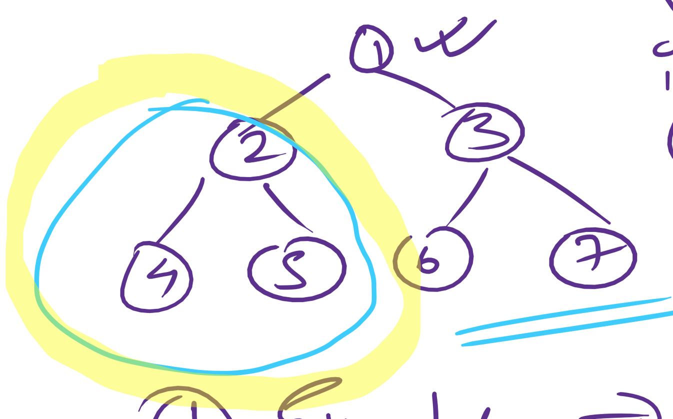
swapping



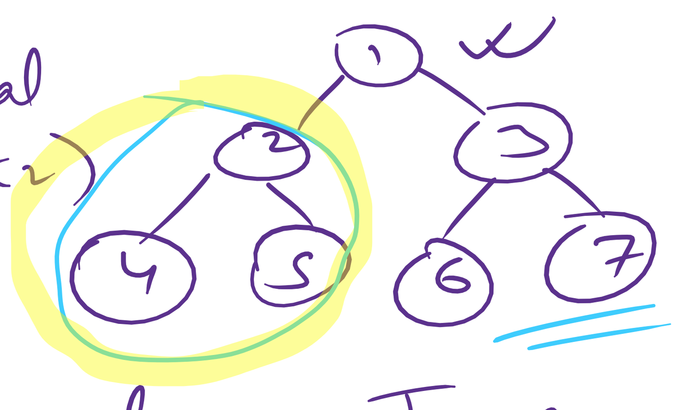
7,3,6,1,5,2,4
In-order

TreeNode* mirrorTree (TreeNode* root) {
* To do operations on subtrees, use recursion.
}

Identical Binary Trees :→



identical
(t_1, t_2)



1. Empty ⇒ Equal → True
2. OEmp O NEmp ⇒ False ||
3. Both NEmp root data != → False

left ↓ recursion right ↓

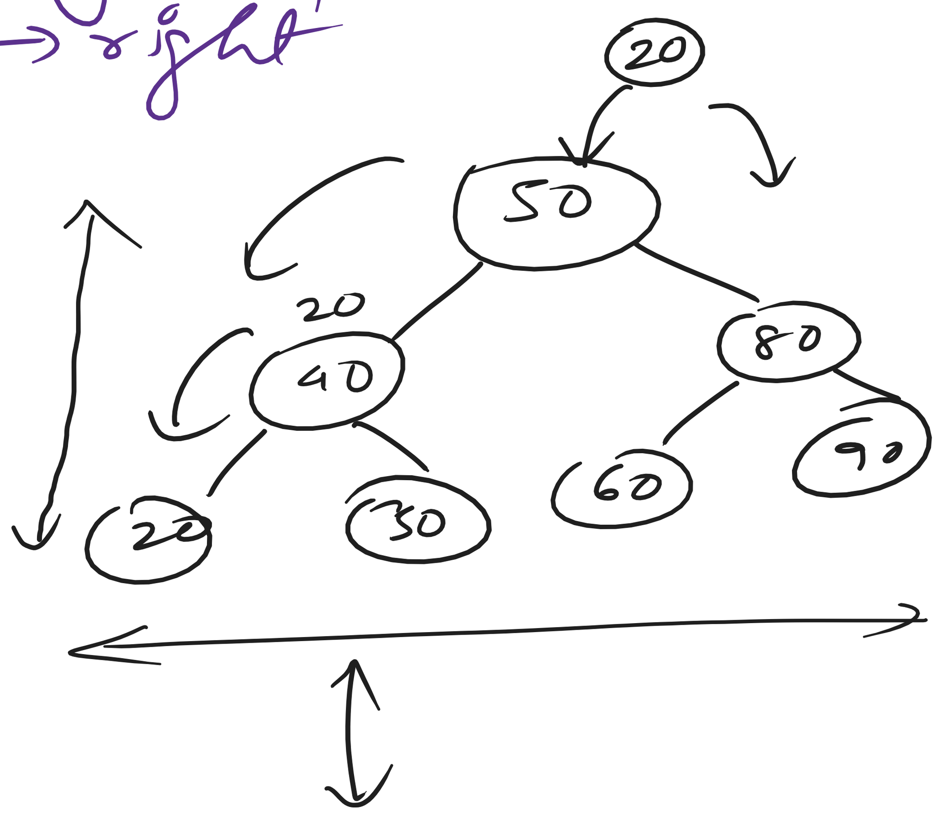
id (t_1 left t_2 left)   id (t_1 r − t_2 r)

Binary Search Tree :→

It is a special kind of binary tree that maintains the following properties:

1. Node → left → right
2. L < N < R

TC → $\log_2 N$ ⇐ $\frac{N}{2}$ $\frac{N}{2}$



1. Insertion
2. Deletion
3. Search
4. Traversal

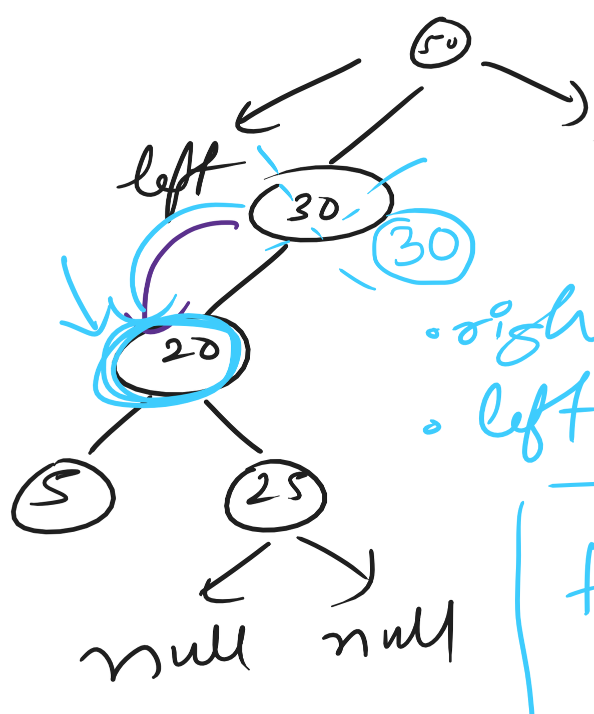
Deletion :→

20

80

30 →

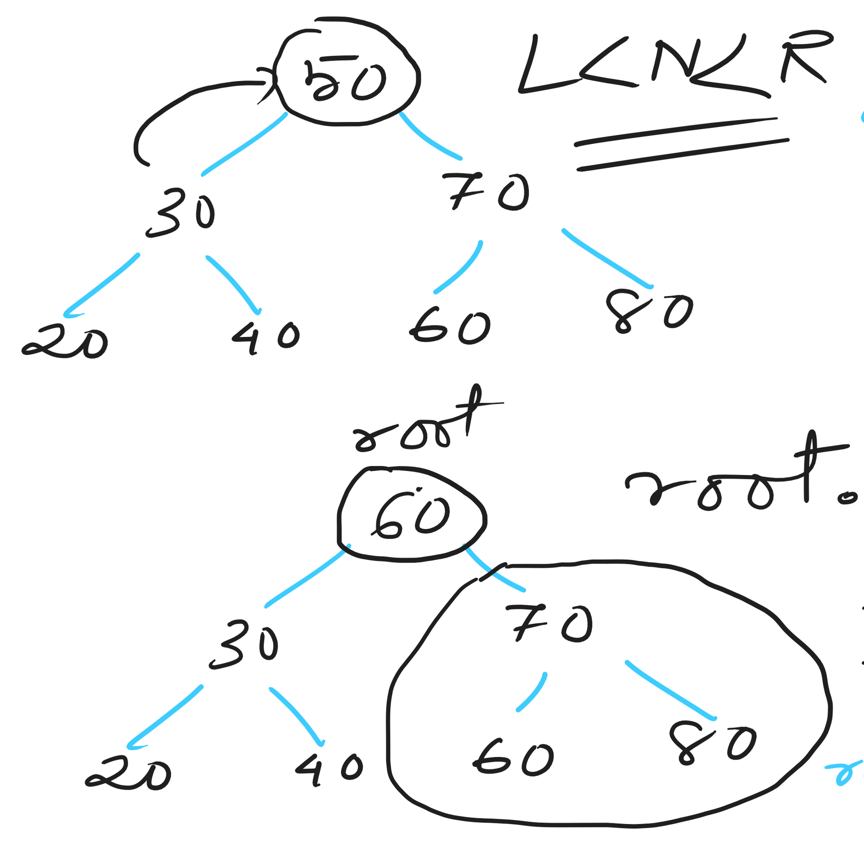
✓ Immediate → 1
* Immediate → 2
leaf ← Immediate → 0
✓ delete



.right = nullptr
.left = nullptr

one child

temp = r. left delete root return temp.	temp = r.right return temp



L<N<R   delete → 50

Leftmost part of right subtree

root. data = finmin (root. right)

temp

finMin (   )

root. right

In-order successor : ✓

20 − 30 − 40 − 50 − 60 − 70 − 80